

Introduction to Databases

Autumn 2025

Exercise 5

(Theory) Hand-in: 10.12.2025 (ADAM, 23:59)
(Practical) Hand-in: 10.12.2025 (during exercise)

Solving the Exercises: The exercises can be solved in small groups of a maximum of two people. Use the notations introduced in the lecture. The DMI plagiarism guidelines apply for this lecture.

Submission Information: Please upload all (Theory) deliverables BEFORE the deadline to ADAM as a **single PDF**. For (Practical) exercise check the **Hand-In** instructions. The solutions must contain your names. Solutions that are handed in too late cannot be considered.

Task (Practical) 1: Importing data from spreadsheets (3 Points)

Go to this BFS page¹ and download the Excel file called “Gemeindestand”. This Excel file contains a table of all municipalities of Switzerland. Sound familiar?

Your job is now to create an SQLite database from the Excel file following these instructions.

When you look at the Excel table, you will notice that many rows contain redundant information (for example the same district id and district name pairs are present in multiple rows). This is considered bad form in database design and we want a normalized schema (why this is bad and normalization are topics of the Database Systems course). As we have not covered normalization yet, here is the normalized relational schema:

canton	(canton_short, canton_name)
district	(district_id, distric_name, canton_short)
municipality	(municipality_id, municipality_name, hist_nr, join_date, district_id)

Mark the primary and foreign keys in the normalized relational schema.

Now use the normalized relational schema to write the corresponding **CREATE TABLE** statements. In addition to the primary and foreign key constraints, you must also add appropriate unique and nullability constraints to the columns. Choose the type of the column based on the data in the Excel table.

¹<https://www.agvchapp.bfs.admin.ch/de>

Create a SQLite database with these tables and find a way to import the data from the Excel table into this SQLite database. What method did you use?

Hint: The contents for the `canton_name` column in the `canton` table are not available in the Excel file, you must find a way another way to fill that column.

In order to show that you did not lose any data in the conversion, write an SQL query that produces the original content of the table (with matching column names).

Optional: Did you know that you can display the virtual machine code that SQLite uses to execute the query? If you put **EXPLAIN** before the query in the SQLite shell, it will provide the virtual machine code in a human readable form. You can look up the description of the opcodes in the documentation². Modify the query from last week to work with the SQLite database. Execute the query with **EXPLAIN** and compare the steps of the virtual machine code with the steps of your visualization.

Hand-In: Show the database schema, conversion process and the query to one of the assistants/tutors.

²https://sqlite.org/opcode.html#the_opcodes

Task (Practical) 2: A web interface for the chess database (4 Points)

In this task you will modify a web application so that users can filter for certain games in the chess database.

On ADAM you find a template for the web application written in Python using Flask. Download the template.

Adjust the `DATABASE_PATH` variable in `app.py` to the SQLite database created in exercise 3. By default it expects the database to be called `lichess.db` and being in the same directory as `app.py`.

Run the web application with `uvx` like this:

```
uvx flask --app ex5.py run --reload
```

Now visit the following location with a web browser: `http://127.0.0.1:5000`.

As you can see it does not retrieve any data (yet). First write a query that retrieves the required fields (`game_id`, `game_start`, name of the white player, name of the black player, `opening_text` and the `result`) in that order without any filters. Create a view for that query in the SQLite database with the `sqlite3` tool and use it in your solution. Put the `CREATE VIEW` statement into the placeholder in `app.py`

Hint: You can use `strftime('%Y-%m-%d %H:%M', game_start, 'unixepoch')` to format the date.

Fill in the three `if` statements in `app.py` to modify the query when the filter fields are not empty. For the player name the user enters a `LIKE` pattern which should be matched against either player name. The opening field should also be treated as a `LIKE` pattern. The result field should be compared directly.

Look up what SQL injections are and ensure that your solution is not vulnerable.

All parts that need modification are marked with `# Modify`.

You can put `LIMIT 25` at the end of your queries to limit the number of results.

Hand-In: Show the web interface to one of the assistants/tutors and upload the modified `app.py` to ADAM.

Task (Theory) 3: Integrity

(2 Points)

You are tasked with completing the design of a table which stores budget request for an accounting firm. For this you are given the following boilerplate for a `CREATE TABLE` statement:

```
CREATE TABLE request (  
    title VARCHAR(255) [...],  
    applicant VARCHAR(255) [...],  
    reviewer1 VARCHAR(255) [...],  
    reviewer2 VARCHAR(255) [...],  
    amount INTEGER [...],  
    [...]  
);
```

Additionally, you are also given the following table, which holds all people.

```
CREATE TABLE people (  
    f_name VARCHAR(255),  
    l_name VARCHAR(255),  
    id VARCHAR(255) NOT NULL,  
    PRIMARY KEY(id)  
);
```

Complete the `CREATE TABLE` statement `request` and ensure that the following constraints hold:

- Each request requires an `applicant` and `amount`.
- A request is identified by its `applicant` and its `title`.
- `reviewer1`, `reviewer2` and `applicant` are people.
- The requested `amount` should be between 200 and 500'000.
- The `title` cannot be empty.
- Each row requires at least a `reviewer1`, if the amount is larger than 50'000 also a `reviewer2` is required.
- The applicant can not also be a `reviewer` for the same request.
- If there are 2 `reviewers` they must be distinct.

Hand-In: Upload the resulting `CREATE TABLE` statement to ADAM.

Task (Theory) 4: NoSQL

(1 Points)

For this exercise you are presented with multiple data management scenarios and possible databases/model to deal with them. Pick the most suitable Database type (*Relational Database, Key-Value Store, Column Store, Document Database, Graph Database*), explain your decision shortly (1-2 sentences) and provide a rough approach how you would store the data in the database (1-3 sentences). Each database type has to be used at least once.

1. Infrastructure of air, railway and vehicle network.
2. Bookkeeping system which heavily relies on aggregating the stored values.
3. List of Students, Exercises and how many points they achieved for each of them.
4. Log files with different content collected from network switches.
5. Website cache, which stores the results of expensive payloads.
6. Museum storing metadata about its collection of art, with attributes varying between exhibits.

Hand-In: Upload your solution to ADAM.